Crio.Do

LEARNING ROADMAP

Low Level Design (LLD) Roadmap

Object-Oriented Design for Interviews & Real Systems

Intensive	Balanced	Flexible
20-25 hrs/week	10-15 hrs/week	5-8 hrs/week

▪ 7 Phases ★ Intermediate to Advanced

https://crio.do

Overview

Low Level Design is about structuring code so it's maintainable, extensible, and testable. Unlike High Level Design (which asks 'what components exist?'), LLD asks 'how do these classes work together?' It's tested in interviews at Amazon, Google, Microsoft, and most product companies. This roadmap takes you from OOP fundamentals to confidently designing systems like parking lots, elevators, and chess games — the classic LLD interview questions. But more importantly, it teaches you to write code that doesn't become legacy the moment you finish it.

Who is this for?

Software developers preparing for FAANG/product company interviews, developers who want to write better-structured code, tech leads who design systems for teams, or anyone who's written code they're embarrassed to maintain.

Prerequisites

- Proficiency in at least one OOP language (Java, Python, C++, C#)
- Ability to write classes, methods, and basic data structures
- Understanding of basic programming concepts
- Ideally 6+ months of coding experience
- Familiarity with any codebase you've had to modify

What you'll learn

- Design classes and relationships that are easy to extend
- Apply SOLID principles naturally, not as a checklist
- Choose the right design pattern for the problem
- Draw UML diagrams that communicate your design
- Solve classic LLD interview problems (parking lot, elevator, chess)
- Approach any LLD interview with a clear framework
- Write code that other developers thank you for

Learning Plans

Choose a plan that fits your schedule. All plans cover the same content.

Intensive

Fast Track

20-25 hrs/week

100-125 total hrs

Focused preparation for upcoming interviews. Practice multiple problems weekly with peer review.

Ideal for:

- Interview prep (4-6 weeks out)
- Career changers targeting FAANG
- Developers between jobs
- Those with dedicated study time

Balanced

Recommended

10-15 hrs/week

120-180 total hrs

Steady learning while working. Apply concepts to your day job. The recommended approach for long-term skill building.

Ideal for:

- Working developers
- Those improving code quality at work
- Long-term interview prep
- Tech leads learning design

Flexible

Self-paced

5-8 hrs/week

150-200 total hrs

Gradual mastery with time to internalize concepts. Best for deep understanding over speed.

Ideal for:

- Busy professionals
- Those learning design for the first time
- Weekend learners
- No immediate interview pressure

Phase Schedule by Plan

Phase	Topic	Intensive	Balanced	Flexible
1	OOP Foundations	Week 1	Weeks 1-2	Weeks 1-4
2	SOLID Principles	Week 2	Weeks 3-4	Weeks 5-8
3	Design Patterns	Weeks 3-4	Weeks 5-8	Weeks 9-16
4	UML & System Modeling	Week 4	Weeks 9-10	Weeks 17-20
5	LLD Interview Framework	Week 5	Weeks 11-12	Weeks 21-24
6	Classic LLD Problems	Week 6	Weeks 13-16	Weeks 25-32
7	Advanced Topics & Interview Mastery	Week 7	Weeks 17-20	Weeks 33-40

1 OOP Foundations

Intensive: Week 1 | Balanced: Weeks 1-2 | Flexible: Weeks 1-4

Before patterns and principles, you need rock-solid OOP fundamentals. These aren't just interview topics — they're the building blocks of every design decision. Most developers know the definitions but struggle to apply them correctly. This phase ensures you understand not just what abstraction is, but when and why to use it.

Learning Objectives

- Explain and apply the four pillars of OOP
- Design classes with proper encapsulation
- Use inheritance appropriately (and know when not to)
- Implement polymorphism for flexible designs
- Understand abstraction as a design tool
- Recognize relationships: association, aggregation, composition

Topics to Cover

Classes and Objects

WHY: Everything in OOP is built on classes and objects. Understanding the difference between a class (blueprint) and object (instance) is fundamental. Getting this wrong leads to confusion about state, behavior, and memory.

- Classes as blueprints
- Objects as instances
- Constructors and initialization
- Static vs instance members
- Object lifecycle
- Identity vs equality

Encapsulation

WHY: Encapsulation hides internal state and requires all interaction through well-defined interfaces. It prevents bugs from 'reaching into' objects and changing things directly. Good encapsulation makes code predictable.

- Private state, public interface
- Getters and setters (use sparingly)
- Information hiding
- Immutability as encapsulation
- Tell, don't ask principle
- Encapsulation at class boundaries

Inheritance

WHY: Inheritance models 'is-a' relationships and enables code reuse. But misused inheritance creates fragile hierarchies. Understanding when inheritance fits — and when composition is better — is a senior skill.

- Superclass and subclass
- Method overriding
- The 'is-a' test
- Protected vs private
- Fragile base class problem
- When NOT to use inheritance

Polymorphism

WHY: Polymorphism lets you write code that works with objects of different types. It's the foundation of extensible systems — add new types without changing existing code. Most design patterns rely on polymorphism.

- Method overriding (runtime polymorphism)
- Method overloading (compile-time)
- Interfaces and polymorphism
- Programming to interfaces
- Dynamic dispatch
- Polymorphism in design patterns

Abstraction

WHY: Abstraction means exposing only what's necessary. Abstract classes and interfaces define contracts without implementation details. Good abstraction lets you change implementation without affecting clients.

- Abstract classes vs interfaces
- When to use each
- Defining contracts
- Hiding complexity
- Levels of abstraction
- Leaky abstractions

Object Relationships

WHY: Objects don't exist in isolation. Understanding association (uses), aggregation (has-a, shared lifetime), and composition (has-a, owned lifetime) helps you model real-world relationships correctly.

- Association (uses-a)
- Aggregation (has-a, shared)
- Composition (has-a, owned)
- Dependency (depends-on)
- Multiplicity (1:1, 1:n, n:m)
- Bidirectional vs unidirectional

Hands-on Projects

Shape Hierarchy

Design a shape system: Shape (abstract), Circle, Rectangle, Triangle. Each has area() and perimeter(). Add a Canvas that holds shapes and calculates total area. Focus on proper use of abstraction and polymorphism.

Abstraction | Polymorphism | Inheritance

Library System (Basic)

Design classes for: Book (title, author, ISBN), Member (name, id, borrowed books), Library (collection of books, members). Model the relationships correctly. A book can be borrowed by one member; a member can borrow multiple books.

Class design | Relationships | Encapsulation

Resources

BOOK **Head First Object-Oriented Analysis & Design**

TUTORIAL **OOP Concepts (Oracle Java Tutorials)**

TUTORIAL **Refactoring Guru - OOP Basics**

Milestone

Design a small system with 5+ classes that have clear relationships. Explain your use of inheritance vs composition. Identify where polymorphism makes your design extensible.

2 SOLID Principles

Intensive: Week 2 | Balanced: Weeks 3-4 | Flexible: Weeks 5-8

SOLID principles are the foundation of maintainable object-oriented design. Introduced by Robert C. Martin, they're not rules to follow blindly but guidelines that lead to flexible, understandable code. Interviewers often ask about SOLID directly, but more importantly, they expect your designs to reflect these principles naturally.

Learning Objectives

- Apply Single Responsibility to keep classes focused
- Use Open-Closed to extend without modifying
- Ensure Liskov Substitution for safe inheritance
- Design interfaces that aren't bloated (Interface Segregation)
- Depend on abstractions, not concretions (Dependency Inversion)
- Recognize SOLID violations in existing code

Topics to Cover

Single Responsibility Principle (SRP)

WHY: 'A class should have only one reason to change.' Classes with multiple responsibilities are hard to test, hard to understand, and change for multiple reasons. SRP keeps classes focused and cohesive.

- One reason to change
- Identifying responsibilities
- Separating concerns
- Cohesion measurement
- Common SRP violations
- Refactoring to SRP

Open-Closed Principle (OCP)

WHY: 'Open for extension, closed for modification.' You should be able to add new functionality without changing existing, tested code. This is achieved through abstraction and polymorphism.

- Extension vs modification
- Using interfaces for extension
- Strategy pattern and OCP
- Plugin architectures
- When modification is acceptable
- Real-world OCP examples

Liskov Substitution Principle (LSP)

WHY: 'Subtypes must be substitutable for their base types.' If your Square class extends Rectangle but breaks when used as a Rectangle, you've violated LSP. This principle ensures inheritance hierarchies are meaningful.

- Substitutability requirement
- The Rectangle-Square problem
- Behavioral compatibility
- Preconditions and postconditions
- Design by contract
- Fixing LSP violations

Interface Segregation Principle (ISP)

WHY: 'Clients should not be forced to depend on interfaces they don't use.' Fat interfaces force implementers to provide methods they don't need. Smaller, focused interfaces are more flexible and easier to implement.

- Fat interfaces problem
- Role interfaces
- Splitting interfaces
- Client-specific interfaces
- Interface pollution
- ISP and microservices

Dependency Inversion Principle (DIP)

WHY: 'Depend on abstractions, not concretions.' High-level modules shouldn't depend on low-level modules. Both should depend on abstractions. This enables swapping implementations and makes testing easy.

- Abstractions vs concretions
- Dependency injection
- Inversion of control
- Constructor injection
- Interface-based design
- Testing with DIP

Hands-on Projects

Refactor a Violation

Take a 'God class' that handles user management, email sending, and logging. Refactor it to follow SRP — separate into focused classes. Then apply DIP by injecting dependencies through interfaces.

SRP · DIP · Refactoring · Dependency injection

Payment System Design

Design a payment system that supports credit cards. Then extend it to support PayPal and crypto WITHOUT modifying existing code (OCP). Use interfaces and dependency injection. Ensure new payment types are substitutable (LSP).

OCP · LSP · DIP · Interface design

Resources

TUTORIAL **SOLID Principles (Baeldung)**

BOOK **Clean Architecture (Robert Martin)**

TUTORIAL **SOLID Principles in Python (Real Python)**

Milestone

Review a codebase (yours or open source) and identify SOLID violations. Refactor at least one class to fix violations. Explain each principle with a real example, not textbook definitions.

3 Design Patterns

Intensive: Weeks 3-4 | Balanced: Weeks 5-8 | Flexible: Weeks 9-16

Design patterns are proven solutions to common problems. The Gang of Four catalogued 23 patterns in 1994, and they're still relevant today. But patterns aren't meant to be forced — they emerge when you recognize a problem they solve. This phase teaches you to recognize when a pattern fits, not to use patterns for their own sake.

Learning Objectives

- Recognize problems that patterns solve
- Implement key creational patterns (Factory, Singleton, Builder)
- Apply structural patterns (Adapter, Decorator, Facade)
- Use behavioral patterns (Strategy, Observer, Command)
- Know when NOT to use patterns
- Combine patterns in real designs

Topics to Cover

Creational Patterns

WHY: Creational patterns control how objects are created. Direct instantiation (new) creates tight coupling. These patterns abstract the creation process, making systems more flexible.

- Singleton (and why it's often an anti-pattern)
- Factory Method
- Abstract Factory
- Builder
- Prototype
- When to use each

Factory Pattern Deep Dive

WHY: Factory encapsulates object creation logic. When you need to create objects without specifying exact classes, or when creation logic is complex, Factory is your tool. It's one of the most commonly used patterns.

- Simple Factory
- Factory Method pattern
- Abstract Factory pattern
- Factory vs Constructor
- Parameterized factories
- Factory in frameworks

Structural Patterns

WHY: Structural patterns compose objects into larger structures. They help you build flexible systems by defining how classes and objects connect.

- Adapter (convert interfaces)
- Decorator (add behavior dynamically)
- Facade (simplify complex subsystems)
- Composite (tree structures)
- Proxy (control access)
- Bridge (separate abstraction from implementation)

Strategy Pattern Deep Dive

WHY: Strategy encapsulates algorithms and makes them interchangeable. Instead of hardcoding behavior, you inject it. This is fundamental to OCP — add new algorithms without changing existing code.

- Encapsulating algorithms
- Strategy vs inheritance
- Context and strategy relationship
- Runtime algorithm switching
- Strategy in validation/pricing
- Strategy vs Template Method

Observer Pattern Deep Dive

WHY: Observer defines a one-to-many dependency. When one object changes, all dependents are notified. It's the foundation of event-driven systems, UI frameworks, and reactive programming.

- Subject and observers
- Push vs pull notification
- Event handling
- Loose coupling through Observer
- Observer in MVC
- Observer vs Pub-Sub

Other Essential Patterns

WHY: Beyond the deep dives, several patterns appear frequently in interviews and real systems. Know these well enough to recognize and apply them.

- Command (encapsulate requests)
- State (state-dependent behavior)
- Template Method (algorithm skeleton)
- Chain of Responsibility
- Null Object
- Repository pattern

Hands-on Projects

Notification System

Build a notification system supporting Email, SMS, and Push notifications. Use Factory to create notifiers. Use Strategy for notification scheduling (immediate, batched, scheduled). Use Observer for event subscriptions.

Factory | Strategy | Observer | Pattern combination

File Processing Pipeline

Design a file processor that reads files, applies transformations (compression, encryption, formatting), and outputs results. Use Decorator for transformations (stack them). Use Strategy for output formats. Use Template Method for the processing skeleton.

Decorator | Strategy | Template Method | Composition

Resources

TUTORIAL **Refactoring Guru - Design Patterns**

BOOK **Head First Design Patterns**

RESOURCE **Design Patterns for Humans (GitHub)**

Milestone

Implement Factory, Strategy, and Observer from memory. Given a problem description, identify which pattern(s) apply. Explain the trade-offs of using Singleton.

4 UML & System Modeling

Intensive: Week 4 | Balanced: Weeks 9-10 | Flexible: Weeks 17-20

UML (Unified Modeling Language) is a visual language for design. While you won't use all 14 diagram types, class diagrams and sequence diagrams are essential for communicating designs in interviews and with teams. A well-drawn diagram often communicates more than pages of code.

Learning Objectives

- Draw class diagrams that communicate structure
- Create sequence diagrams for interactions
- Use proper UML notation for relationships
- Know when diagrams help vs waste time
- Whiteboard designs clearly in interviews

Topics to Cover

Class Diagrams

WHY: Class diagrams show the static structure of a system — classes, attributes, methods, and relationships. In LLD interviews, you'll often sketch class diagrams before coding. They help you think through design before committing to code.

- Class notation (name, attributes, methods)
- Visibility markers (+, -, #)
- Relationships (association, aggregation, composition)
- Inheritance and interface implementation
- Multiplicity notation
- Drawing quickly on whiteboards

Sequence Diagrams

WHY: Sequence diagrams show how objects interact over time. They're perfect for visualizing method calls, API flows, and complex interactions. When asked 'how does X work?', a sequence diagram is often the clearest answer.

- Objects and lifelines
- Messages and returns
- Self-calls
- Loops and conditionals
- Creating and destroying objects
- Common interaction patterns

Other Useful Diagrams

WHY: Beyond class and sequence, a few other diagrams occasionally help. State diagrams for state machines, activity diagrams for workflows. Know when they add value.

- State diagrams (for state machines)
- Activity diagrams (for workflows)
- Use case diagrams (for requirements)
- Component diagrams (for architecture)
- When to skip UML
- Informal diagrams that work

Whiteboard Techniques

WHY: In interviews, you draw on whiteboards, not UML tools. Speed and clarity matter more than perfect notation. Practice drawing quickly while explaining your design.

- Drawing while talking
- Simplified notation for interviews
- Organizing whiteboard space
- Using colors effectively
- Iterating on diagrams
- Common interview mistakes

Hands-on Projects

Diagram an Existing System

Take a system you've built or use daily (a simple app, library, or feature). Draw its class diagram showing main classes and relationships. Draw a sequence diagram for a key operation. Practice explaining it in 5 minutes.

Class diagrams | Sequence diagrams | Communication

Design from Diagram

Look at a class diagram for a system you haven't seen. Implement it in code without other documentation. This tests whether your diagrams are complete enough to code from.

Reading UML | Implementation | Design completeness

Resources

BOOK **UML Distilled (Martin Fowler)**

TOOL **PlantUML (Text-based diagrams)**

TUTORIAL **Lucidchart UML Guide**

Milestone

Draw a class diagram for any system in under 5 minutes. Draw a sequence diagram for a complex interaction. Your diagrams should be clear enough that another developer could implement from them.

5 LLD Interview Framework

Intensive: Week 5 | Balanced: Weeks 11-12 | Flexible: Weeks 21-24

LLD interviews follow a pattern. You get a vague prompt ('Design a parking lot'), and you need to turn it into a working design. Most candidates fail by jumping straight to code. This phase teaches you a systematic framework that keeps you organized and demonstrates senior-level thinking.

Learning Objectives

- Follow a consistent framework for any LLD problem
- Ask the right clarifying questions
- Identify entities and relationships systematically
- Manage interview time effectively
- Handle follow-up questions gracefully
- Communicate design decisions clearly

Topics to Cover

The 5-Phase Framework

WHY: A framework prevents cognitive overload. Instead of thinking about everything at once, you handle each phase sequentially. This makes your approach predictable and demonstrates structured thinking.

- Phase 1: Clarify Requirements (2 min)
- Phase 2: Identify Core Entities (5 min)
- Phase 3: Design Classes & Relationships (10 min)
- Phase 4: Implement Key Methods (15 min)
- Phase 5: Handle Extensions (5 min)
- Time management strategies

Clarifying Requirements

WHY: The prompt is intentionally vague. Jumping in without questions means you're guessing. Good clarifying questions show you think like a product-minded engineer and scope the problem appropriately.

- Primary capabilities
- Boundary conditions
- Scale and constraints
- Error handling expectations
- What's in vs out of scope
- Sample clarifying questions

Identifying Entities

WHY: Entities become classes. Missing an entity means missing a class. Identifying entities systematically ensures your design is complete. Look for nouns in requirements.

- Extracting nouns from requirements
- Core vs supporting entities
- Enums for fixed categories
- Entity relationships
- Avoiding premature optimization
- Entity vs value object

Designing Class APIs

WHY: Once you have entities, define their interfaces. What methods does each class expose? What's the contract? Good API design makes implementation straightforward and communicates intent.

- Identifying key methods
- Method signatures
- Return types and exceptions
- Public vs private interface
- SOLID in API design
- API naming conventions

Handling Extensions

WHY: Interviewers test flexibility by asking 'what if we add X?' Your design should handle extensions without major restructuring. This is where SOLID and patterns pay off.

- Common extension questions
- Designing for extensibility
- Adding new types (OCP)
- Adding new operations
- Graceful degradation
- Knowing your design's limits

Hands-on Projects

Timed Practice Sessions

Set a 35-minute timer. Pick a problem you haven't solved. Follow the framework strictly: 2 min requirements, 5 min entities, 10 min classes, 15 min implementation, 3 min review. Repeat with different problems.

Time management | Framework application | Pressure handling

Mock Interviews

Practice with a peer or mentor. Have them play interviewer — ask clarifying questions, push back on decisions, request extensions. Get feedback on communication and design quality.

Communication | Handling pressure | Thinking aloud

Resources

GUIDE **Hello Interview - LLD Delivery Framework**

ARTICLE **AlgoMaster - How to Answer LLD Problems**

PRACTICE **Pramp (Free mock interviews)**

Milestone

Complete 5 LLD problems using the framework within 35 minutes each. Your designs should naturally apply SOLID and relevant patterns. You should be able to explain every design decision.

6 Classic LLD Problems

Intensive: Week 6 | Balanced: Weeks 13-16 | Flexible: Weeks 25-32

Certain problems appear repeatedly in interviews: parking lot, elevator, chess, library management, and more. This phase covers the most common ones. But the goal isn't to memorize solutions — it's to practice applying everything you've learned to diverse domains.

Learning Objectives

- Solve classic LLD problems confidently
- Recognize patterns across different domains
- Handle variations and extensions
- Build a mental library of design approaches
- Adapt solutions to interviewer's requirements

Topics to Cover

Parking Lot System

WHY: The classic LLD interview question. Tests entity modeling, relationships, and state management. Involves: ParkingLot, Floor, Spot (types), Vehicle (types), Ticket, Payment. Common patterns: Factory (spots), Strategy (pricing).

- Core entities
- Spot types and vehicle types
- Entry/exit flow
- Pricing strategies
- Capacity management
- Common extensions

Elevator System

WHY: Tests state management, scheduling algorithms, and concurrent requests. Involves: Elevator, Floor, Request, Controller. Common patterns: State (elevator states), Strategy (scheduling), Command (requests).

- Core entities
- Request types (internal/external)
- Scheduling algorithms
- Multiple elevators
- State transitions
- Optimization considerations

Library Management System

WHY: Tests CRUD operations, relationships, and business rules. Involves: Library, Book, Member, Loan, Fine. Good for practicing: many-to-many relationships, temporal logic, business rules.

- Core entities
- Book-Member relationships
- Loan and return flow
- Fine calculation
- Search functionality
- Reservation system

Chess / Tic-Tac-Toe

WHY: Tests game modeling, move validation, and turn management. Chess is more complex; Tic-Tac-Toe is simpler but covers same concepts. Involves: Board, Piece (types), Player, Move, Game.

- Board representation
- Piece hierarchy
- Move validation
- Game state (check, checkmate)
- Turn management
- Undo functionality

Online Booking Systems

WHY: Movie tickets, hotels, flights — all follow similar patterns. Tests inventory management, concurrent booking, and payment integration. Involves: Venue, Show/Room, Seat, Booking, Payment.

- Inventory modeling
- Seat selection
- Concurrent booking handling
- Booking states
- Cancellation and refund
- Notification system

Social / E-commerce Systems

WHY: More complex systems with many entities. Tests your ability to scope appropriately and focus on core functionality. Usually simplified versions in interviews.

- User and relationships
- Content/Product modeling
- Feed generation
- Cart and checkout
- Search and recommendations
- Scoping for interviews

Hands-on Projects

Solve All Classic Problems

Work through: Parking Lot, Elevator, Library, Tic-Tac-Toe, and one booking system. For each: identify entities, draw class diagram, implement core methods, handle 2 extensions. Don't look at solutions until you've tried.

All LLD skills | Pattern recognition | Diverse domains

Create Your Own Problem

Think of a system you use daily (food delivery, ride sharing, messaging). Design it from scratch. This tests whether you can apply the framework to novel problems, not just memorized ones.

Requirements analysis | Original thinking | Complete design

Resources

RESOURCE **Awesome Low Level Design (GitHub)**

COURSE **Grokking the Low Level Design Interview**

PRACTICE **LLD Interview Questions (GeeksforGeeks)**

Milestone

Solve any classic LLD problem in 35 minutes without hints. Your solutions should apply SOLID principles and appropriate patterns. Be able to handle common extensions confidently.

7 Advanced Topics & Interview Mastery

Intensive: Week 7 | Balanced: Weeks 17-20 | Flexible: Weeks 33-40

Senior candidates face harder questions: concurrency, design trade-offs, real-world constraints. This phase covers advanced topics and polishes your interview performance. The goal is to move from 'can solve the problem' to 'would hire confidently.'

Learning Objectives

- Handle concurrency in LLD problems
- Discuss trade-offs articulately
- Connect LLD to system design
- Demonstrate senior-level thinking
- Build a portfolio of LLD solutions

Topics to Cover

Concurrency Considerations

WHY: Real systems have concurrent users. Parking lots have multiple entry points. Elevators handle simultaneous requests. Understanding thread safety, locking, and race conditions is a senior skill.

- Identifying concurrent scenarios
- Thread safety basics
- Synchronization strategies
- Lock granularity
- Avoiding deadlocks
- When to mention concurrency

Design Trade-offs

WHY: There's no perfect design. Trade-offs between flexibility, performance, and simplicity are constant. Articulating trade-offs shows mature engineering judgment.

- Flexibility vs simplicity
- Performance vs maintainability
- Premature optimization
- When to use patterns vs not
- Technical debt awareness
- Communicating trade-offs

LLD to HLD Connection

WHY: LLD doesn't exist in isolation. Understanding how your classes map to services, databases, and APIs shows system-level thinking. Some interviews blend LLD and HLD.

- Classes to services
- Persistence considerations
- API design from classes
- Scaling implications
- When LLD becomes HLD
- Full-stack thinking

Code Quality & Testing

WHY: Great designs are testable. If you can't test a class easily, it's probably poorly designed. Mentioning testing in interviews demonstrates quality consciousness.

- Designing for testability
- Dependency injection for testing
- Unit vs integration testing
- Test-driven design
- Mentioning testing in interviews
- Code review perspective

Building Your Portfolio

WHY: A GitHub with well-documented LLD solutions demonstrates your skills better than certifications. Interviewers sometimes review your code before interviews.

- Documenting solutions
- Clean code in portfolio
- README quality
- Showing design evolution
- Highlighting pattern usage
- Interview preparation from portfolio

Hands-on Projects

Concurrent Parking Lot

Extend your parking lot design to handle concurrent entry/exit. Multiple cars entering simultaneously shouldn't assign the same spot. Implement proper synchronization. Discuss trade-offs.

Concurrency | Synchronization | Trade-off analysis

LLD Portfolio

Create a GitHub repository with 10+ LLD solutions. Each should have: problem statement, requirements, class diagram, code, and a README explaining design decisions. Make it interview-ready.

Documentation | Portfolio building | Communication

Resources

BOOK **Designing Data-Intensive Applications**

BOOK **Java Concurrency in Practice**

RESOURCE **System Design Primer (GitHub)**

Milestone

Handle any LLD interview confidently. Discuss concurrency and trade-offs when relevant. Have a portfolio that demonstrates your skills. Get offers from target companies.

Your Path Forward

Career Opportunities

- ✓ Pass LLD interviews at FAANG and top companies
- ✓ Design maintainable systems at work
- ✓ Communicate designs effectively to teams
- ✓ Review others' designs constructively
- ✓ Progress to senior/staff engineering roles

Tips for Success

- Don't memorize solutions — understand the principles behind them
- Practice drawing while talking; it's a skill that needs work
- Time yourself; 35 minutes goes fast in interviews
- Explain trade-offs, not just solutions
- Start simple, then add complexity; don't over-engineer initially
- Ask clarifying questions; it's a sign of experience, not weakness
- Know your design patterns, but don't force them
- SOLID principles should feel natural, not like a checklist
- Review your old code; seeing violations helps you avoid them
- Mock interviews are invaluable — practice with others

Next Steps

1. Review OOP in your primary language
2. Pick one SOLID principle and find violations in your codebase
3. Solve one classic LLD problem this week
4. Draw a class diagram for a system you built
5. Schedule a mock interview with a peer
6. Create a GitHub repo for your LLD practice

Ready to Start Your Journey?

Everything in this roadmap is free to learn on your own. But if you'd like structured guidance, hands-on projects, and mentor support — we'd love to have you at Crio.Do

Explore Crio.Do

https://crio.do